

3장 UI 디자인 및 구현

UI 디자인 및 구현은 Figma 시안을 기반으로 spacing·hierarchy·color 시스템을 이해하고 이를 코드로 변환하는 기준을 마련한 뒤, 버튼·입력필드·카드 같은 기본 UI 컴포넌트를 제작하고 styled-components 및 CSS 모듈로 스타일을 관리하며, 사용자 입력 처리와 상태 관리, 접근성 및 UX 개선을 포함해 완전한 UI 흐름을 구축하는 과정을 다룬다.

3.1 Figma 연계 및 코드 설계

Figma 연계 및 코드 설계는 디자이너와 개발자 간의 협업을 효율화하기 위해 UI 시안을 분석하고, spacing·hierarchy·color 시스템을 정립하며, 디자인 요소를 코드로 변환하는 기준을 마련함으로써 일관된 UI/UX를 보장하고 유지보수성 높은 프론트엔드 구조를 구현하는 과정이다.

애플리케이션 생성

```
npx create-react-app ex01
```

디렉터리 구조

```
src/  
├─ components/  
│  ├─ Button/  
│  │  ├─ Button.jsx  
│  │  ├─ Button.module.css  
│  │  └─ Button.stories.jsx  
│  ├─ TextInput/  
│  └─ Card/  
├─ styles/  
│  ├─ global.css  
│  ├─ tokens.css ← generated  
│  └─ theme.js  
└─ pages/
```

```
└─ tools/  
  └─ tokens-to-css.js
```

(1) Figma 시안 분석

Figma 시안 분석은 디자인 시안을 체계적으로 해석하여 컴포넌트 구조, 속성, 계층을 파악하고 코드로 전환 시 우선순위를 정립해 개발 효율성과 UI 일관성을 확보하는 단계이다.

- ① Figma에서 제공된 UI 시안 가져오기 → 협업 툴에서 최신 시안을 가져와 개발 환경에 연동해 디자인과 코드의 싱크를 유지한다.
- ② 아트보드/프레임 구조 분석 → 페이지 단위 또는 컴포넌트 단위로 프레임 구조를 파악하여 코드 매핑 방식을 명확히 한다.
- ③ 컴포넌트 네이밍 규칙 및 계층 구조 파악 → 일관된 네이밍과 계층 구조를 분석해 재사용성과 협업 효율을 높인다.
- ④ UI 요소별 디자인 속성 추출 → 색상, 폰트, 간격, 크기 등 스타일 속성을 식별해 코드 변수로 반영할 준비를 한다.
- ⑤ 개발 코드 반영 우선순위 정의 → 핵심 UI 요소부터 구현할 대상을 정리해 개발 효율과 출시 일정을 최적화한다.

(2) Spacing, Hierarchy, Color 시스템 이해

Spacing·Hierarchy·Color 시스템 이해는 UI의 공간 배치, 정보 계층, 색상 체계를 표준화하여 다양한 화면 크기와 모드에서도 일관된 디자인 경험을 제공하기 위한 핵심 원칙이다.

- ① Spacing 단위 표준화 → 패딩·마진을 4px 단위 그리드로 규격화해 UI 간격을 체계적이고 일관되게 유지한다.
- ② Hierarchy 계층 구조 분석 → 제목, 본문, 캡션의 시각적 중요도를 구분해 정보 전달력을 강화한다.
- ③ Color system 정의 → 주요 색상군을 Primary, Secondary, Neutral, Accent로 구분해 디자인 가이드라인을 확립한다.
- ④ 다크/라이트 모드 고려 → 두 가지 모드에서 가독성과 접근성을 유지하기 위한 색상 대비 규칙을 적용한다.
- ⑤ 반응형 spacing·hierarchy 조정 → 다양한 디바이스 크기에서 간격과 글자 크기를 유연하게 조정해 최적의 UX를 보장한다.

(3) 디자인 → 코드 변환 기준 정립

디자인 → 코드 변환 기준 정립은 디자인 토큰 정의, 공통 스타일 관리, 컴포넌트 추상화, 레이아웃 시스템 설계, 협업 프로세스를 통해 디자인과 개발 간의 간극을 줄이고 유지보수 가능한 프론트엔드 코드베이스를 완성하는 과정이다.

- ① 디자인 토큰 정의 → 색상, 폰트, spacing 값을 변수화해 디자인과 코드 간의 싱크를 유지한다.
- ② 공통 스타일 관리 → global.css 또는 theme.js를 활용해 전역 스타일을 일관되게 제어한다.
- ③ 컴포넌트 변환 기준 → 버튼, 인풋 같은 최소 단위 요소를 컴포넌트로 추상화해 재사용성을 높인다.
- ④ 레이아웃 시스템 변환 → flex와 grid를 활용해 반응형 레이아웃을 코드로 구현한다.
- ⑤ 협업 프로세스 확립 → Figma → Storybook → 코드 변환의 흐름을 정의해 디자이너·개발자 협업을 원활히 한다.

(4) Figma를 코드화 작업

이번에는 Figma 시안을 코드로 변환하는 작업을 진행하겠습니다.

① Figma 시안 → 코드

1. 디자이너에게서 최신 Figma 파일(또는 Exported JSON) 확보.
2. Figma Inspect 또는 **Figma Tokens** 플러그인으로 색상/폰트/spacing 등 토큰 수집.
3. 토큰을 JSON으로 정리 (`design-tokens.json`).
4. 토큰 → CSS 변수(`:root`) 자동 생성 스크립트 실행(또는 수동 복사).
5. `ThemeProvider` (styled-components)와 `global.css` 에서 변수 사용.
6. 컴포넌트(버튼/인풋/카드) 구현 — props 기반 API 설계.
7. Storybook에 스토리 추가(controls로 variant/size 테스트).
8. QA(접근성, 반응형, 다크모드) → 배포.

② Figma에서 토큰/속성 추출하는 방법

- **수동(Inspect 패널):** 각 레이어를 선택 → 오른쪽 Inspect에서 color/font/spacing 값 복사.
- **자동(Figma Tokens 플러그인 권장):** 플러그인으로 색상/타이포/스페이싱 토큰을 추출 → JSON 내보내기.
- **컴포넌트 매핑:** Figma의 Component/Instance 네이밍을 코드 컴포넌트명(예: `Button/Primary`)으로 맞추기.

③ 디자인 토큰(design-tokens.json)

```
{
  "color": {
    "primary": "#0B74FF",
    "primary-600": "#095FCC",
    "secondary": "#FF7A59",
    "neutral-900": "#0B1220",
    "neutral-700": "#4B5563",
    "neutral-100": "#F3F4F6",
    "accent": "#F59E0B"
  },
  "font": {
    "family": "Inter, system-ui, -apple-system, 'Segoe UI', Roboto, 'Helvetica Neue', Arial",
    "baseSize": "16px",
    "scale": { "sm": "0.875rem", "md": "1rem", "lg": "1.125rem" }
  },
  "spacing": {
    "xs": "4px",
    "sm": "8px",
    "md": "16px",
    "lg": "24px",
    "xl": "32px"
  },
  "radius": { "sm": "4px", "md": "8px", "lg": "12px" },
  "elevation": { "1": "0 1px 2px rgba(16,24,40,0.05)", "2": "0 4px 10px rgba(16,24,40,0.05)" }
```

```

6,24,40,0.08)" },
  "dark": {
    "color": {
      "primary": "#7FB8FF",
      "neutral-900": "#E6EEF8",
      "neutral-100": "#0B1220"
    }
  }
}

```

④ 토큰 → CSS 변수 자동 생성 (Node 스크립트 예)

tools/tokens-to-css.js

```

// node tools/tokens-to-css.js design-tokens.json > src/styles/tokens.css
const fs = require('fs');
const path = require('path');

const file = process.argv[2] || './design-tokens.json';
const tokens = JSON.parse(fs.readFileSync(path.resolve(file), 'utf8'));

function flatten(obj, prefix = '') {
  return Object.entries(obj).flatMap(([k, v]) => {
    const name = prefix ? `${prefix}-${k}` : k;
    if (typeof v === 'object' && !Array.isArray(v)) {
      return flatten(v, name);
    }
    return `--${name}: ${v};`;
  });
}

const root = flatten(tokens).filter(line => !line.startsWith('--dark-'));
const dark = flatten(tokens.dark || {}, 'dark');

console.log(':root {}');
root.forEach(l => console.log(' ' + l));
console.log('');
console.log('');

```

```
console.log('[data-theme="dark"] {}');
dark.forEach(l => console.log(' ' + l.replace('--dark-', '---')));
console.log('}');
```

- 명령: `node tools/tokens-to-css.js design-tokens.json > src/styles/tokens.css`
- 결과: `:root` 와 `[data-theme="dark"]` 에 CSS 커스텀 프로퍼티 생성

⑤ global.css (tokens 사용 예)

`src/styles/global.css`

```
@import './tokens.css';

:root {
  font-size: var(--font-baseSize, 16px);
}

html, body, #root {
  height: 100%;
}

body {
  font-family: var(--font-family);
  background: var(--neutral-100);
  color: var(--neutral-900);
  margin: 0;
  padding: 0;
}

[data-theme="dark"] body {
  background: var(--neutral-100);
  color: var(--neutral-900);
}

/* utility */
.container {
  max-width: 1200px;
  margin: 0 auto;
```

```
padding-left: var(--spacing-md);
padding-right: var(--spacing-md);
}
```

토큰 파일에서 font-family, spacing-md 등 이름을 맞춰두면 CSS에서 바로 참조 가능합니다.

⑥ theme.js (styled-components용)

src/styles/theme.js

```
import tokens from '../design-tokens.json';

const theme = {
  colors: {
    primary: tokens.color.primary,
    secondary: tokens.color.secondary,
    neutral: {
      900: tokens.color['neutral-900'],
      100: tokens.color['neutral-100'],
      700: tokens.color['neutral-700']
    },
    accent: tokens.color.accent
  },
  spacing: tokens.spacing,
  radius: tokens.radius,
  elevation: tokens.elevation,
  fonts: {
    body: tokens.font.family,
    sizes: tokens.font.scale
  }
};

export default theme;
```

- 앱 진입부에서 `ThemeProvider` 로 감싸 사용:

```
import { ThemeProvider } from 'styled-components';
import theme from './styles/theme';
import './styles/global.css';

function AppRoot() {
  return (
    <ThemeProvider theme={theme}>
      <App />
    </ThemeProvider>
  );
}
```

⑦ 컴포넌트 예제 1 — Button (styled-components)

src/components/Button.jsx

```
import React from 'react';
import styled, { css } from 'styled-components';
import clsx from 'clsx';

const sizes = {
  sm: css`padding: 6px 10px; font-size: ${p => p.theme.fonts.sizes.sm};`,
  md: css`padding: 8px 14px; font-size: ${p => p.theme.fonts.sizes.md};`,
  lg: css`padding: 12px 18px; font-size: ${p => p.theme.fonts.sizes.lg};`
};

const variants = {
  primary: css`
    background: ${p => p.theme.colors.primary};
    color: white;
    &:hover { filter: brightness(0.95); }
  `,
  secondary: css`
    background: ${p => p.theme.colors.secondary};
    color: white;
  `,
  ghost: css`
```



```

background: transparent;
color: ${p ⇒ p.theme.colors.primary};
border: 1px solid ${p ⇒ p.theme.colors.primary};
,
};

const StyledButton = styled.button`
border: 0;
border-radius: ${p ⇒ p.theme.radius.md};
cursor: pointer;
display: inline-flex;
align-items: center;
justify-content: center;
${p ⇒ sizes[p.size || 'md']};
${p ⇒ variants[p.variant || 'primary']};
opacity: ${p ⇒ (p.disabled ? 0.6 : 1)};
pointer-events: ${p ⇒ (p.disabled ? 'none' : 'auto')};
&[aria-busy="true"] { cursor: progress; }
`;

export default function Button({ children, variant='primary', size='md', loading=false, disabled=false, ...rest }) {
  return (
    <StyledButton variant={variant}
      size={size}
      disabled={disabled}
      aria-busy={loading}
      {...rest}
    >
      {loading ? '로딩...' : children}
    </StyledButton>
  );
}

```

⑧ 컴포넌트 예제 2 — Button (CSS Modules)

src/components/Button.module.css

```
.btn { border: 0; border-radius: 8px; cursor: pointer; display: inline-flex; align-items: center; justify-content: center; }
.btn.primary { background: var(--primary); color: white; }
.btn.secondary { background: var(--secondary); color: white; }
.btn.sm { padding: 6px 10px; font-size: 0.875rem; }
.btn.md { padding: 8px 14px; font-size: 1rem; }
.btn.lg { padding: 12px 18px; font-size: 1.125rem; }
.btn.disabled { opacity: 0.6; pointer-events: none; }
```

src/components/ButtonModule.jsx

```
import React from 'react';
import styles from './Button.module.css';
import cs from 'classnames';

export default function ButtonModule({ children, variant='primary', size='md', disabled=false, ...rest }) {
  const className = cs(styles.btn, styles[variant], styles[size], { [styles.disabled]: disabled });
  return <button className={className} disabled={disabled} {...rest}>{children}</button>;
}
```

⑨ Input 컴포넌트 예제 (유효성, ARIA)

src/components/TextInput.jsx

```
import React, { useState, useEffect } from 'react';

export default function TextInput({ label, value, onChange, placeholder, required=false, pattern, id }) {
  const [touched, setTouched] = useState(false);
  const [error, setError] = useState('');

  useEffect(() => {
    if (!touched) return;
    if (required && !value) setError('필수 입력입니다.');
```

```

않습니다.');
```

```

    else setError('');
  }, [value, touched, required, pattern]);

  return (
    <div className="form-row">
      {label && <label htmlFor={id}>{label}{required && '*'}</label>}
      <input id={id}
        value={value}
        placeholder={placeholder}
        onChange={e => onChange(e.target.value)}
        onBlur={() => setTouched(true)}
        aria-invalid={!!error}
        aria-describedby={error ? `${id}-error` : undefined}
      />
      {error && <div id={`${id}-error`} role="alert" style={{ color: 'red' }}>{error}</div>}
    </div>
  );
}

```

⑩ Card 컴포넌트 (구조화)

src/components/Card.module.css

```

.card { background: white; border-radius: 8px; box-shadow: var(--elevation
-1); padding: var(--spacing-md); }
.card-header { font-weight: 600; margin-bottom: 8px; }
.card-body { margin-bottom: 12px; }
.card-footer { border-top: 1px solid #eee; padding-top: 8px; text-align: right; }

```

src/components/Card.jsx

```

import React from 'react';
import styles from './Card.module.css';

```

```
export default function Card({ header, footer, children }) {
  return (
    <div className={styles.card}>
      {header} && <div className={styles['card-header']}>{header}</div>
      <div className={styles['card-body']}>{children}</div>
      {footer} && <div className={styles['card-footer']}>{footer}</div>
    </div>
  );
}
```

⑪ 레이아웃 시스템 샘플 (flex/grid)

src/components/Layout.css

```
.row { display:flex; gap:var(--spacing-md); flex-wrap:wrap; }
.col { flex:1 1 0; min-width: 220px; }
.grid-2 { display:grid; grid-template-columns: repeat(2, 1fr); gap: var(--spacing-lg); }
@media (max-width: 600px) { .grid-2 { grid-template-columns: 1fr; } }
```

⑫ Storybook 예시 (Button.stories.jsx)

```
import React from 'react';
import Button from '../components/Button';

export default { title: 'UI/Button', component: Button, argTypes: { variant: { control: 'select', options: ['primary','secondary','ghost'] }, size: { control: 'select', options: ['sm','md','lg'] } } };

const Template = args => <Button {...args}>버튼</Button>;

export const Primary = Template.bind({});
Primary.args = { variant: 'primary', size: 'md' };

export const Loading = Template.bind({});
Loading.args = { variant: 'primary', size: 'md', loading: true };
```

⑬ Figma 속성 → 코드 �핑 표

- Figma **Fill** (Color) → `-color-primary` 또는 `theme.colors.primary`
- Figma **Font Family/Weight/Size/Line Height** → `-font-family`, `font-size`, `line-height` 토큰
- Figma **Spacing** → spacing tokens (`-spacing-md`)
- Figma **Corner Radius** → `-radius-md`
- Shadow → `-elevation-1` (box-shadow value)

3.2 버튼, 입력 필드, 카드 컴포넌트 제작

기본 UI 컴포넌트 제작은 버튼, 입력필드, 카드와 같은 핵심 UI 블록을 구현하고 props 기반 스타일링과 컴포넌트 구조화를 통해 재사용성을 높이며, styled-components 및 CSS 모듈로 스타일 관리 체계를 확립하고 사용자 입력 처리·상태 관리·폼 검증·접근성·UX 개선을 적용해 실제 서비스에 활용 가능한 실무형 UI 기반을 마련하는 과정이다.

디렉터리 구조

```
src/
├─ components/
│  └─ ui/
│     └─ Button.jsx
│     └─ TextInput.jsx
│     └─ Card.jsx
│  └─ css/
│     └─ Button.module.css
│     └─ TextInput.module.css
│     └─ Card.module.css
│  └─ index.js
└─ styles/
```

```
| └─ tokens.css
| └─ App.jsx
└─ main.jsx
```

(1) 버튼, 입력필드, 카드 구현

버튼, 입력필드, 카드 구현은 서비스 전반에서 자주 사용되는 기본 UI 요소를 상태별·구조별로 정의하고 props 설계 전략을 적용하여 확장성과 재사용성을 갖춘 공통 컴포넌트를 구축하는 단계이다.

- ① Button → size, variant, disabled, loading 등 상태별 스타일을 정의해 다양한 버튼 형태를 일관되게 제공한다.
- ② Input → placeholder, validation, focus, error 처리 로직을 포함해 사용자 친화적인 입력 경험을 보장한다.
- ③ Card → header, body, footer 영역을 구조화하고 그림자·보더·라운드 속성을 적용해 정보 표시의 시각적 완성도를 높인다.
- ④ 공통 컴포넌트 props 설계 전략 → 각 컴포넌트에 공통적으로 적용될 props를 정의해 일관성·재사용성을 강화한다.

(2) styled-components 및 CSS 모듈 활용

styled-components와 CSS 모듈 활용은 props 기반 동적 스타일링, ThemeProvider를 통한 전역 디자인 시스템 관리, 클래스 충돌 방지, CSS-in-JS와 모듈 혼합 전략을 통해 유지보수성과 성능을 동시에 확보하는 방식이다.

- ① styled-components → props 값을 활용한 동적 스타일링으로 컴포넌트 UI 변화를 유연하게 처리한다.
- ② ThemeProvider → 색상, 폰트, spacing 등 전역 디자인 시스템을 일관되게 관리할 수 있도록 적용한다.
- ③ CSS 모듈 → 컴포넌트 단위 캡슐화로 클래스 충돌을 방지하고 코드 가독성을 향상시킨다.
- ④ CSS-in-JS와 CSS 모듈 혼합 전략 → CSS-in-JS의 동적 스타일링과 CSS 모듈의 구조화를 결합해 장점을 극대화한다.

- ⑤ 실무 유지보수성/성능 비교 → CSS-in-JS와 CSS 모듈의 장단점을 평가해 프로젝트 요구사항에 맞는 최적의 방식을 선택한다.

(3) 사용자 입력 처리

사용자 입력 처리는 onClick·onChange·onSubmit 이벤트 핸들링과 상태 관리, 폼 검증, 접근성 고려, UX 개선 기능을 포함해 사용자가 자연스럽게 오류 없는 입력 경험을 할 수 있도록 지원하는 과정이다.

- ① 이벤트 처리 → onClick, onChange, onSubmit 이벤트를 정의해 입력·제출 동작을 제어한다.
- ② 상태 관리 → useState, useReducer를 활용해 입력값을 추적하고 동적 UI 갱신을 구현한다.
- ③ 폼 검증 로직 → 필수 입력값 확인, 패턴 체크, 실시간 오류 메시지를 제공해 데이터 정확성을 보장한다.
- ④ 접근성 고려 → label 연결, aria 속성, 키보드 내비게이션을 반영해 접근성을 강화한다.
- ⑤ UX 개선 → 자동 포커스, 입력 시 실시간 피드백 등으로 사용자 편의성과 만족도를 높인다.

(4) 디자인 토큰 / Theme (styled-components 용)

src/theme.js

```
export const theme = {
  colors: {
    primary: '#0B74FF',
    primary700: '#095FCC',
    neutral100: '#F7F7FA',
    neutral300: '#D1D5DB',
    neutral900: '#111827',
    danger: '#E11D48'
  },
  space: {
    xs: '4px',
```

```

    sm: '8px',
    md: '16px',
    lg: '24px'
  },
  radii: { sm: '6px', md: '10px' },
  fonts: { body: 'Inter, system-ui, -apple-system, "Segoe UI", Roboto' }
};

```

(5) styled-components 구현 (권장: 동적 스타일링·Theme 지원)

① Button (styled-components)

src/components/ui/Button.jsx

```

import React from 'react';
import styled, { css } from 'styled-components';

const sizeStyles = {
  sm: css`padding: 6px 10px; font-size: 14px;`,
  md: css`padding: 10px 16px; font-size: 16px;`,
  lg: css`padding: 14px 20px; font-size: 18px;`
};

const variantStyles = {
  primary: css`
    background: ${({theme}) => theme.colors.primary};
    color: white;
    &:hover { background: ${({theme}) => theme.colors.primary700}; }
  `,
  ghost: css`
    background: transparent;
    border: 1px solid ${({theme}) => theme.colors.primary};
    color: ${({theme}) => theme.colors.primary};
  `
};

const StyledButton = styled.button`

```



```

display:inline-flex;
gap:8px;
align-items:center;
border-radius: ${({theme}) => theme.radii.sm};
border: none;
cursor: pointer;
${(p) => sizeStyles[p.size || 'md']}
${(p) => variantStyles[p.variant || 'primary']}
${(p) => p.disabled && `opacity:0.6; cursor:not-allowed;`}
`;

export default function Button({
  children,
  variant = 'primary',
  size = 'md',
  disabled = false,
  loading = false,
  ...rest
}) {
  return (
    <StyledButtonvariant={variant}
      size={size}
      disabled={disabled || loading}
      aria-disabled={disabled || loading}
      aria-busy={loading || undefined}
      {...rest}
    >
      {loading && <span aria-hidden>⌚</span>}
      <span>{children}</span>
    </StyledButton>
  );
}

```

- `loading` 일 때 `aria-busy`, 로딩 아이콘 추가, 버튼 비활성화.
- `variant` / `size` 로 UI 변형 제어.

② TextInput (styled-components, controlled, forwardRef, error support)

```

import React, { forwardRef } from 'react';
import styled from 'styled-components';

const Input = styled.input`
  width: 100%;
  padding: ${({theme}) => theme.space.sm};
  border: 1px solid ${({theme}) => theme.colors.neutral300};
  border-radius: ${({theme}) => theme.radii.sm};
  font-size: 16px;
  &:focus {
    outline: 3px solid rgba(11,116,255,0.15);
    border-color: ${({theme}) => theme.colors.primary700};
  }
  &[aria-invalid="true"] {
    border-color: ${({theme}) => theme.colors.danger};
  }
`;

const Error = styled.div`
  color: ${({theme}) => theme.colors.danger};
  font-size: 13px;
  margin-top: 6px;
`;

const Wrapper = styled.div`display:flex; flex-direction:column; gap:6px;`;

export default forwardRef(function TextInput(
  { id, label, value, onChange, placeholder, type='text', error, ...rest },
  ref
) {
  return (
    <Wrapper>
      {label} && <label htmlFor={id}>{label}</label>
      <Input id={id}
        ref={ref}
        value={value}
        onChange={(e) => onChange?.(e.target.value)}

```

```

placeholder={placeholder}
type={type}
aria-invalid={!error}
aria-describedby={error ? `${id}-error` : undefined}
{...rest}
/>
{error && <Error id={`${id}-error`} role="alert">{error}</Error>}
</Wrapper>
);
});

```

- `onChange` 는 문자열만 넘기도록 설계(단순화).
- `aria-invalid` , `aria-describedby` 로 접근성 제공.
- `forwardRef`로 포커스 관리 가능.

③ Card (styled-components)

src/components/ui/Card.jsx

```

import React from 'react';
import styled from 'styled-components';

const Container = styled.div`
  background: white;
  border-radius: ${({theme}) => theme.radii.md};
  box-shadow: 0 1px 6px rgba(16,24,40,0.08);
  border: 1px solid #e6e9ee;
  overflow: hidden;
`;

const Header = styled.div`padding: 16px; border-bottom: 1px solid #f3f4f6; font-weight: 600;`;
const Body = styled.div`padding: 16px;`;
const Footer = styled.div`padding: 12px; border-top: 1px solid #f3f4f6; background: #fafafa;`;

export default function Card({ header, footer, children }) {
  return (

```

```

    <Container role="region" aria-label={header || 'card'}>
      {header && <Header>{header}</Header>}
      <Body>{children}</Body>
      {footer && <Footer>{footer}</Footer>}
    </Container>
  );
}

```

(6) CSS Modules 구현 (정적/레이아웃 중심)

CSS Modules를 선호하면 스타일을 CSS 파일로 유지하고 className으로 제어하세요. 동적 상태(variant 등)엔 classnames(또는 간단한 조건문) 사용 권장.

① Button.module.css

```

.btn {
  border-radius: 6px;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  cursor: pointer;
}
.primary { background: #0B74FF; color: white; }
.ghost { background: transparent; border: 1px solid #0B74FF; color: #0B74FF; }
.sm { padding: 6px 10px; font-size: 14px; }
.md { padding: 10px 16px; font-size: 16px; }
.lg { padding: 14px 20px; font-size: 18px; }
.disabled { opacity: 0.6; cursor: not-allowed; }

```

② Button (CSS Modules)

src/components/css/ButtonCss.jsx

```
import React from 'react';
import styles from './Button.module.css';

export default function ButtonCss({ children, variant='primary', size='md',
disabled=false, ...rest }) {
  const className = [
    styles.btn,
    styles[variant],
    styles[size],
    disabled ? styles.disabled : ''
  ].join(' ');
  return (
    <button className={className} disabled={disabled} {...rest}>
      {children}
    </button>
  );
}
```

(7) CSS-in-JS vs CSS Modules 혼합 전략

- **권장 전략**
 - **styled-components:** 동적 스타일(variant, theme, animations)과 ThemeProvider 기반 디자인 시스템.
 - **CSS Modules:** 레이아웃, 글로벌 유틸리티, 성능 민감한 정적 스타일(대규모 리스트)에서 사용.
- **이유:** CSS-in-JS는 런타임 비용(비교적)과 클래스 생성 오버헤드가 있지만 props 기반 동적 스타일이 쉬움. CSS Modules는 빌드 시 결합된 CSS로 런타임 비용 적음.

(8) Props 설계 전략

- **Button props**
 - `variant?: 'primary' | 'ghost' | 'danger'`

- `size?: 'sm' | 'md' | 'lg'`
- `disabled?: boolean`
- `loading?: boolean`
- `as?: string | Component` (polymorphic)
- `onClick?: (e) ⇒ void`

• TextInput props

- `id`, `name`, `value`, `defaultValue`, `onChange(value)`, `placeholder`, `type`, `required`, `pattern`, `error`

• Card props

- `header`, `footer`, `children`, `variant?`, `shadow?`, `rounded?`

원칙: boolean props엔 `is` / `has` 접두사를 권장하지 않는 대신 `disabled` 같은 표준명을 쓰고 이벤트는 항상 `onX` 형식을 유지하세요.

(9) 사용자 입력 처리: 이벤트·state·검증 예제

① 간단한 Controlled Form (useState)

`src/examples/LoginFormSimple.jsx`

```
import React, { useState } from 'react';
import TextInput from '../components/ui/TextInput';
import Button from '../components/ui/Button';

export default function LoginFormSimple() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleSubmit = async (e) ⇒ {
    e.preventDefault();
    setError(null);
    if (!email || !password) return setError('모든 필드를 채워주세요');
    setLoading(true);
  }
}
```

```

try {
  // fake API
  await new Promise(res => setTimeout(res, 1000));
  alert('로그인 성공: ' + email);
} catch (err) {
  setError('로그인 실패');
} finally {
  setLoading(false);
}
};

return (
  <form onSubmit={handleSubmit} aria-live="polite">
    <TextInput id="email" label="이메일" value={email} onChange={setEmail} placeholder="you@example.com" />
    <TextInput id="password" label="비밀번호" value={password} onChange={setPassword} type="password" />
    {error && <div role="alert" style={{color:'red'}}>{error}</div>}
    <Button type="submit" loading={loading}>로그인</Button>
  </form>
);
}

```

② 복잡한 폼: useReducer + validation

src/examples/LoginFormReducer.jsx

```

import React, { useReducer, useRef } from 'react';
import TextInput from '../components/ui/TextInput';
import Button from '../components/ui/Button';

const initialState = { values: { email:'', password:'' }, errors: {}, touched: {}, loading: false, serverError: null };

function reducer(state, action) {
  switch(action.type) {
    case 'SET_FIELD': return { ...state, values: { ...state.values, [action.field]: action.value } };
  }
}

```

```

    case 'SET_ERROR': return { ...state, errors: { ...state.errors, [action.field]:
action.error } };
    case 'SET_TOUCHED': return { ...state, touched: { ...state.touched, [action.field]: true } };
    case 'SUBMIT_START': return { ...state, loading: true, serverError: null };
    case 'SUBMIT_SUCCESS': return { ...state, loading: false };
    case 'SUBMIT_FAIL': return { ...state, loading: false, serverError: action.error };
    default: return state;
  }
}

```

```

function validate({ email, password }) {
  const errors = {};
  if (!email) errors.email = '이메일을 입력하세요';
  else if (!/^\S+@\S+\.\S+$/.test(email)) errors.email = '유효한 이메일이 아닙니다';
  if (!password) errors.password = '비밀번호를 입력하세요';
  else if (password.length < 6) errors.password = '비밀번호는 6자 이상';
  return errors;
}

```

```

export default function LoginFormReducer() {
  const [state, dispatch] = useReducer(reducer, initialState);
  const emailRef = useRef();

  const handleChange = (field) => (val) => {
    dispatch({ type: 'SET_FIELD', field, value: val });
    const errors = validate({ ...state.values, [field]: val });
    dispatch({ type: 'SET_ERROR', field, error: errors[field] });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const errors = validate(state.values);
    if (Object.keys(errors).length) {
      Object.keys(errors).forEach(f => dispatch({ type: 'SET_ERROR', field: f, error: errors[f] }));
    }
  };
}

```



```

    return;
  }
  dispatch({ type: 'SUBMIT_START' });
  try {
    await new Promise((res) => setTimeout(res, 900)); // fake API
    dispatch({ type: 'SUBMIT_SUCCESS' });
    alert('로그인 성공');
  } catch (err) {
    dispatch({ type: 'SUBMIT_FAIL', error: '서버 오류' });
  }
};

return (
  <form onSubmit={handleSubmit} noValidate>
    <TextInput id="email" ref={emailRef} label="이메일" value={state.value
s.email} onChange={handleChange('email')} error={state.errors.email} />
    <TextInput id="password" label="비밀번호" value={state.values.passwo
rd} onChange={handleChange('password')} type="password" error={stat
e.errors.password} />
    {state.serverError && <div role="alert" style={{color:'red'}}>{state.serv
erError}</div>}
    <Button type="submit" loading={state.loading}>로그인</Button>
  </form>
);
}

```

- `validate()` 는 동기 검증(클라이언트). 서버 에러는 별도 `serverError` .
- `noValidate` 로 브라우저 기본 폼 검증 방지(커스텀 검사 사용 시).
- `aria-live` / `role="alert"` 로 스크린리더에게 에러 알림.

(10) 접근성(A11y) 권장사항

- 각 `input` 에 대해 `<label htmlFor>` 또는 `aria-label` 제공.
- 오류 메시지에는 `role="alert"` 및 `aria-describedby` 로 연결.
- 버튼의 로딩 상태에 `aria-busy` 또는 `aria-disabled` .

- 키보드 네비게이션: `tabindex`, focus 스타일 제공(visible focus ring).
- 색 대비(색상 토큰)를 WCAG AA 기준 이상으로 유지.

(11) UX 개선

- 자동 포커스: 첫 입력에 `autoFocus` 또는 `ref.focus()` 사용.
- 입력 피드백: 입력 중 유효성 검사를 `debounce(300ms)`로 제공.
- 비활성화/로딩: 폼 제출 시 모든 입력 비활성화 및 스피너 표시.
- 에러 포커싱: 제출 시 첫 번째 오류 필드로 포커스 이동.
- Remember Me: 보안·UX 고려 (로컬 저장 유무 명확화).

간단한 debounce 훅

```
import { useState, useEffect } from 'react';
export function useDebounceValue(value, delay = 300) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setV(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return v;
}
```

(12) 성능 & 유지보수 비교

- styled-components
 - 장점: props 기반 동적 스타일, theme 지원, 컴포넌트 단위 캡슐화, JS에서 조건 제어 용이.
 - 단점: 런타임 오버헤드, SSR/스타일 추출 고려 필요.
- CSS Modules

- 장점: 빌드 타임에 클래스 이름 생성 → 런타임 비용 적음, 파일 분리로 유지보수 용이.
- 단점: 동적 스타일(variant)에서 클래스 조합 코드가 번거로움.
- **권장:** 디자인 시스템(토큰, Theme)을 기반으로 하되, **동적 스타일은 styled-components**, **대형 정적 레이아웃·리스트는 CSS Modules**로 혼합 사용.

(13) 샘플 App 연결 (React 19, createRoot)

src/main.jsx

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import { ThemeProvider } from 'styled-components';
import { theme } from './theme';
import './styles/tokens.css';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <ThemeProvider theme={theme}>
      <App />
    </ThemeProvider>
  </React.StrictMode>
);
```

src/App.jsx

```
import React from 'react';
import LoginFormReducer from './examples/LoginFormReducer';
import { Container } from './components/ui/Layout';
import Card from './components/ui/Card';

export default function App() {
  return (
    <Container>
      <Card header="로그인">
```

```

    <LoginFormReducer />
  </Card>
</Container>
);
}

```

3.3 로그인 화면 구현

로그인 화면 구현은 사용자 인증의 핵심 플로우를 설계하고 보안성을 고려한 입력 처리와 상태 관리를 적용하며, 전역 상태 공유를 통해 안전하고 일관된 로그인 경험을 제공하고, UI 배치·반응형 설계·스토리북 시연·결과 검증까지 포함해 실무에서 활용 가능한 완성도 높은 인증 화면을 구현하는 과정이다.

```

ex03/
├── node_modules/
├── public/
│   └── index.html
├── src/
│   ├── data/
│   │   └── member.json
│   ├── components/
│   │   ├── Header.js
│   │   ├── Footer.js
│   │   └── Loader.js
│   ├── pages/
│   │   ├── Login.js
│   │   └── Home.js
│   ├── App.js
│   ├── index.js
│   └── App.css

```

```
|— package.json
|— README.md
```

(1) 로그인 플로우 설계

로그인 플로우 설계는 아이디와 비밀번호 입력을 기본으로 추가 기능과 보안 요소를 반영해 사용자가 직관적이고 안전하게 로그인할 수 있도록 전체 인증 흐름을 정의하는 단계이다.

- ① 로그인 폼 구성 → 아이디와 비밀번호 입력 필드를 중심으로 직관적인 로그인 폼을 구성한다.
- ② Remember Me 등 추가 기능 → 자동 로그인, 비밀번호 찾기, 회원가입 링크를 배치해 사용자 편의성을 높인다.
- ③ 보안 고려 → 비밀번호 마스킹, 입력 제한, 에러 처리 UX로 안전성과 신뢰성을 강화한다.
- ④ 플로우 다이어그램 정의 → 로그인 성공/실패 시의 동작을 시각적으로 정의해 개발·테스트 기준을 명확히 한다.

(2) 상태관리 및 입력 처리

상태관리 및 입력 처리는 React 상태 관리 혹은 API 연동을 통해 입력값을 추적하고, 로딩·오류·성공 흐름을 제어하며 JWT 토큰을 안전하게 관리하고 전역 인증 상태로 공유해 일관된 인증 경험을 제공한다.

- ① 입력 값 관리 → useState와 useReducer로 아이디·비밀번호 입력값을 제어한다.
- ② API 연동 흐름 → onSubmit 이벤트로 로그인 API를 호출하고 응답을 처리한다.
- ③ 로딩/오류/리다이렉트 → 로딩 상태와 오류 메시지를 표시하고 성공 시 페이지를 전환한다.

(3) UI 배치와 결과 시연

UI 배치와 결과 시연은 헤더·메인·푸터의 화면 레이아웃을 구성하고 반응형 디자인과 디자인 시스템을 적용하 컴포넌트 시연과 최종 실행 결과를 검증하는 단계이다.

- ① 레이아웃 구성 → Header, Main(Login Form), Footer로 UI를 구조화한다.

- ② 반응형 배치 → 모바일·데스크탑 환경에 맞는 반응형 레이아웃을 적용한다.
- ③ 디자인 시스템 적용 → spacing, color, hierarchy를 반영해 일관된 스타일을 유지한다.

(4) 설치 및 실행 명령

프로젝트를 생성한 후 필요한 라이브러리를 설치하고, 개발 서버 실행 명령을 통해 실행해봅니다.

```
# 1. 프로젝트 생성
npx create-react-app ex03
cd ex03

# 2. React Router 설치
npm install react-router-dom

# 3. 개발 서버 실행
npm start
```

member.json (src/data/member.json)

```
[
  {
    "id": "user1",
    "password": "pass1234",
    "name": "홍길순"
  },
  {
    "id": "user2",
    "password": "abcd1234",
    "name": "김기태"
  }
]
```

(5) 공통 컴포넌트

공통 컴포넌트는 모든 화면에서 항상 출력되는 컴포넌트로 Header.js, Footer.js, Loader.js 등을 작성합니다.

① src/components/Loader.js

```
function Loader() {  
  return <div className="loader">로딩 중...</div>;  
}  
  
export default Loader;
```

② Header.js

```
function Header() {  
  return (  
    <header>  
      <h1>ex03 로그인 화면</h1>  
    </header>  
  );  
}  
  
export default Header;
```

③ Footer.js

```
function Footer() {  
  return <footer>© 2025 Company Name</footer>;  
}  
  
export default Footer;
```

(6) 페이지 컴포넌트

페이지 컴포넌트는 화면이 전환되면 교체되는 내용이 있는 컴포넌트로 페이지(pages) 디렉터리에 위치하는 컴포넌트를 말하며, Login.js, Home.js 가 있습니다.

① Login.js

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import Loader from "../components/Loader";
import memberData from "../data/member.json";

function Login() {
  const [userId, setUserId] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();
    setLoading(true);
    setError(null);

    // 간단한 로그인 검증
    setTimeout(() => {
      const user = memberData.find(
        (m) => m.id === userId && m.password === password
      );
      if (user) {
        localStorage.setItem("loginUser", JSON.stringify(user));
        navigate("/home");
      } else {
        setError("로그인 실패: 아이디 또는 비밀번호가 잘못되었습니다.");
      }
      setLoading(false);
    }, 1000); // 1초 딜레이로 로딩 표현
  };

  return (
```



```

<main className="login-main">
  {loading && <Loader />}
  <form onSubmit={handleSubmit} className="login-form">
    <input type="text"
      placeholder="아이디"
      value={userId}
      onChange={(e) => setUserId(e.target.value)}
      required
    />
    <input type="password"
      placeholder="비밀번호"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      required
    />
    <div className="options">
      <label>
        <input type="checkbox" /> 자동 로그인
      </label>
      <a href="#">비밀번호 찾기</a>
    </div>
    <button type="submit">로그인</button>
    {error && <p className="error">{error}</p>}
  </form>
</main>
);
}

export default Login;

```

- 입력값 상태 관리 (`useState`)
- member.json에서 아이디/비밀번호 검증
- 로딩/오류 처리
- 성공 시 localStorage에 로그인 사용자 저장 후 홈 화면 이동

② Home.js

```

import React from "react";
import { useNavigate } from "react-router-dom";

function Home() {
  const navigate = useNavigate();
  const user = JSON.parse(localStorage.getItem("loginUser"));

  const handleLogout = () => {
    localStorage.removeItem("loginUser");
    navigate("/");
  };

  return (
    <div className="home">
      <h2>환영합니다, {user ? user.name : "게스트"}!</h2>
      <p>로그인 성공 화면입니다.</p>
      <button onClick={handleLogout}>로그아웃</button>
    </div>
  );
}

export default Home;

```

- 로그인 성공 후 환영 메시지 표시
- 로그아웃 기능 구현

(7) 애플리케이션 관련 요소 작성

애플리케이션을 구성하고, 실행에 필요한 파일에는 App.js, index.js, App.css 파일이 있습니다.

① App.js

```

import { BrowserRouter as Router, Routes, Route } from "react-router-dom";

```

```

import Header from "./components/Header";
import Footer from "./components/Footer";
import Login from "./pages/Login";
import Home from "./pages/Home";
import "./App.css";

function App() {
  return (
    <Router>
      <Header />
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/home" element={<Home />} />
      </Routes>
      <Footer />
    </Router>
  );
}

export default App;

```

② index.js

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./App.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);

```

③ App.css

```

body {
  font-family: Arial, sans-serif;
  margin: 0;
}

```

```
padding: 0;
background: #f0f2f5;
}

header {
background: #0078d4;
color: white;
text-align: center;
padding: 10px;
}

footer {
text-align: center;
padding: 10px;
background: #e5e5e5;
}

.login-main {
display: flex;
justify-content: center;
align-items: center;
height: 80vh;
}

.login-form {
background: white;
padding: 30px;
border-radius: 10px;
display: flex;
flex-direction: column;
gap: 15px;
width: 300px;
}

.login-form input {
padding: 8px;
border: 1px solid #ccc;
border-radius: 4px;
```

```

}

.login-form button {
  padding: 10px;
  background: #0078d4;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.login-form .options {
  display: flex;
  justify-content: space-between;
  font-size: 0.85rem;
}

.login-form a {
  text-decoration: none;
  color: #0078d4;
}

.error {
  color: red;
}

.loader {
  text-align: center;
  margin-bottom: 10px;
}

.home {
  text-align: center;
  margin-top: 50px;
}

```